ARCHITECTURE & PIPELINE DIAGNOSTIC REPORT

OpenShorts — Real Clipping Failure Investigation

Complete Root Cause Analysis, Staging Artifact Inspection & Safest Architectural Repair Plan

INVESTIGATING MODEL	TARGET SYSTEM	RUNTIME ENVIRONMENT	INVESTIGATION DATE
Gemini 4 Argon	OpenShorts / FluxClip	Node.js / Express / FFmpeg	November 10, 2026

1. EXECUTIVE SUMMARY & PROBLEM CLASSIFICATION

Comprehensive technical inspection of the actual staged files in `/tmp/fluxclip-openshorts-staging/uploads/`, the FFprobe binary metadata parser, and backend pipeline execution paths reveals that **Case 1** and **Case 2** represent **two distinct, independent failure points** in different stages of the media ingestion pipeline:

- **Case 1 (Local Upload Failure):** Occurs during **transcript timestamp boundary validation** in `validateMandatoryPipelinePayloads()`. Both local media files uploaded and probed with 100% precision. The error is caused by a hardcoded `+ 1000ms` grace window that rejects standard caption display hold durations lingering 1.48s after dialogue.
- **Case 2 (YouTube URL Failure):** Occurs during **initial media stream resolution** in `resolveSafeExternalMediaUrl()`. The system resolves the YouTube watch URL into an HTML webpage (`text/html`) and passes it directly into `probeMediaFile()`, causing FFprobe to fail when attempting to parse HTML text as a video stream.

2. CASE 1 INVESTIGATION — LOCAL VIDEO UPLOAD

> **Reported Error:** `Transcript cue #251 ends at 00:14:30.240, exceeding media duration.`
> **Inputs:** Local MP4 video, SRT transcript file, Master audio file (MP3/WAV).

Investigation Question	Verified Technical Finding & Evidence
1. Actual video duration according to FFprobe?	**`868,752 ms` (00:14:28.752)** Inspected actual uploaded staging file: `upl_mwasgtos_odnvrc_404479.mp4` (25,815,702 bytes). Format container duration: `868.752000 s`. Video stream 0 (518x360): 868.666s. Audio stream 1: 868.751s. Video stream 2 (1280x720): 868.752s.
2. Actual master audio duration according to FFprobe?	**`868,704 ms` (00:14:28.704)** Inspected actual uploaded staging file: `upl_mwasjjpr_j2n3xp_ARPIT_BALA_GANJEDI_SLANDER....mp3` (17,428,306 bytes). Format container duration: `868.704000 s`. Stream 0 audio duration: `868.704000 s`.
3. Why does transcript cue #251 exceed media duration?	In `src/utils/pipelineLogic.ts` (line 495): `const outOfBoundsCue = parsedTranscript.cues.find((c) => c.endMs > effectiveDurationMs + 1000);` `effectiveDurationMs` is `max(868752, 868704) = 868,752 ms`. The check rejected any cue exceeding `868,752 + 1000 = 869,752 ms` (00:14:29.752). Cue #251 ends at **`870,240 ms`** (00:14:30.240), exceeding the strict 1-second margin by only **488 ms**.
4. Is the transcript from a different source or duration detection incorrect?	**Neither.** Both media files and transcript belong to the exact same 14m 28.75s source. In automated transcription engines (Whisper, YouTube Auto-Subs, ASR engines), the final subtitle cue commonly incorporates a **1.5s–3.0s caption hold** past the final spoken syllable for on-screen readability, or reflects trailing silence trimmed during container muxing.
5. Is the 2.5s A/V sync validation working correctly?	**PASS** **Working correctly.** Drift between video (`868,752 ms`) and master audio (`868,704 ms`) is exactly **`48 ms`**. This is well within the `2,500 ms` (`AV_SYNC_TOLERANCE_MS`) sync threshold and passed validation without issue.

3. CASE 2 INVESTIGATION — YOUTUBE URL RESOLUTION

> **Reported Error:** `FFprobe failed to inspect media: Command failed: /usr/bin/ffprobe ...`
> **Inputs:** YouTube URL (watch or youtu.be), SRT transcript file, Master audio file.

Investigation Question	Verified Technical Finding & Evidence
1. Why is FFprobe unable to inspect this YouTube source?	In `server.ts` (line 1599 & 1601), `resolveSafeExternalMediaUrl(rawUrl)` follows HTTP redirects and returns the YouTube watch URL (`https://www.youtube.com/watch?v=...`). FFprobe is executed directly against this URL: `/usr/bin/ffprobe -v error -select_streams v:0 -show_entries ... https://www.youtube.com/watch?v=...` The YouTube server returns an **HTML document** (`Content-Type: text/html`). FFprobe attempts to demux HTML as binary media and crashes with `Invalid data found when processing input` .
2. Is the URL resolver producing a real, accessible media stream?	**No.** `resolveSafeExternalMediaUrl()` in `server.ts` (lines 476–550) is strictly an SSRF protection and HTTP redirect resolver. It does not extract direct video stream URLs (e.g., `googlevideo.com/videoplayback...`) from YouTube watch pages.
3. Is FFmpeg/FFprobe receiving a valid media URL?	**No.** Both `probeMediaFile()` (lines 1000, 1601) and `renderViralClipWithFfmpeg()` receive the HTML watch page URL, which cannot be decoded or demuxed by FFmpeg.
4. YouTube access restriction, unsupported format, or implementation bug?	**Implementation Bug + Infrastructure Constraint:** 1. **Implementation bug:** `server.ts` routes `youtube_url` into the same direct-stream probing logic as raw `.mp4` URLs without resolving the underlying media stream. 2. **Environment constraint:** `yt-dlp` is not installed in the container environment, and YouTube actively restricts datacenter IPs with bot checks / cipher requirements for direct playback streams.
5. Can authorized YouTube sources be supported reliably?	FFprobe and FFmpeg cannot natively demux YouTube watch pages. To support them reliably, the backend requires a dedicated stream extraction/downloading layer (via `yt-dlp` or media proxy). Where datacenter IP restrictions prevent headless extraction, the pipeline must provide clear, actionable validation rather than an unhandled FFprobe crash.

4. AFFECTED CODE LOCATIONS

File Path	Line Numbers	Component & Defect Description
`src/utils/pipelineLogic.ts`	Lines 493–506	`validateMandatoryPipelinePayloads()` : Overly restrictive `+ 1000ms` cue `endMs` check rejects legitimate final caption display hold buffers (1.5s–3.0s).
`src/utils/pipelineLogic.test.ts`	Lines 110–135	Test assertions enforce the rigid 1,000ms ceiling, requiring alignment with realistic caption drift specifications.
`server.ts`	Lines 476–550	`resolveSafeExternalMediaUrl()` : Lacks `Content-Type` verification and does not distinguish media streams from HTML pages.
`server.ts`	Lines 994–1005, 1598–1609	Job creation and pipeline execution pass raw YouTube HTML URLs directly to `probeMediaFile()` and `ffmpeg` .

5. SAFEST ARCHITECTURAL REPAIR PLAN

> **Part 1: Transcript Trailing Tolerance (Case 1)**

> **Part 2: Stream Validation & YouTube Handling (Case 2)**

- **Preserve A/V Sync:** Retain the `2,500ms` A/V synchronization check between audio and video without modification.
- **Separate Speech Start from Caption Hold:**
  - Reject any cue whose **`startMs`** begins after media duration (`startMs >= effectiveDurationMs + 1000`).
  - Allow cue **`endMs`** to extend up to `effectiveDurationMs + 5000ms` (5-second tolerance) to accommodate natural caption hold and outro buffers.
- **Zero Mutation:** Do not trim or alter the user's transcript cues. FFmpeg rendering bounds are clamped to physical container limits during clip extraction.

- **Content-Type Verification:** In `resolveSafeExternalMediaUrl()` , inspect the terminal `Content-Type` header. If a URL returns `text/html` for a media source, reject it immediately before invoking FFprobe.
- **Dedicated YouTube Resolution:** Route `youtube_url` through a dedicated resolver that extracts or stages the media stream using verified extractors.
- **Actionable Error Handling:** If automated stream extraction is blocked by YouTube datacenter policies, return a clean, descriptive error code (`YOUTUBE_STREAM_EXTRACTION_BLOCKED`) directing the user to provide a direct MP4 or upload the local media file.